

SECAV-O

CODE INSPECTION GUIDE

Preparation, Execution, Reporting, and Development Coach Observation

Working Guide for Structured Software Engineering Coaching

Purpose	Provide a repeatable procedure for conducting code inspections and using inspection evidence for engineering coaching.
Scope	Source code changes, modules, pull requests, defect fixes, refactorings, and AI-assisted code.
Primary Roles	Author / Engineer, Inspector(s), Moderator or Inspection Lead, Development Coach.
Primary Outputs	Inspection record, defect log, action items, inspection report, coaching observations, and follow-up evidence.

1. Purpose and Inspection Principles

A code inspection is a structured engineering review of source code and related work products. Its objective is to identify defects, risks, deviations from requirements or design intent, maintainability problems, security concerns, test gaps, and evidence of engineering competency before those issues propagate to later stages or production.

The inspection is not a performance-rating meeting. The coach uses the inspection to observe engineering behavior, evidence, reasoning, and improvement opportunities. Findings should therefore be separated into: (a) product defects, (b) process observations, and (c) coaching observations.

Principle	Application
Evidence first	Base findings on code, requirements, design decisions, tests, standards, review records, and observable behavior.
Defect over debate	Record the defect or risk first. Long design debates should be moved to a separate technical discussion if they exceed the inspection timebox.
Author is not the defect	Findings describe the work product, not the person.
Human accountability	AI-assisted code is inspected under the same engineering accountability as manually written code; the engineer remains responsible for validation.
Traceability	Whenever possible, link findings to requirement, design, acceptance criteria, coding rule, security practice, or test expectation.
Actionable outcomes	Each material finding has an owner, disposition, and follow-up evidence.

2. Roles and Responsibilities

Role	Responsibilities
Author / Engineer	Provides the code and context, explains intended behavior when requested, answers factual questions, records rework, and provides closure evidence.
Inspector / Reviewer	Prepares independently, identifies defects and risks, records evidence, and participates in the inspection without turning the session into live redesign.
Moderator / Inspection Lead	Confirms readiness, manages scope and timebox, keeps the meeting focused, ensures findings are recorded, and assigns disposition and follow-up.
Development Coach	Observes engineering competency and review behavior, distinguishes product defects from competency gaps, records evidence-based coaching notes, and plans targeted follow-up.

3. Inspection Procedure

Recommended workflow: Select scope -> Confirm readiness -> Prepare independently -> Execute inspection -> Classify and assign findings -> Rework -> Verify closure -> Report -> Coaching follow-up.

3.1 Select and Bound the Inspection

Define exactly what will be inspected. Prefer a bounded change set: pull request, module, defect fix, feature slice, refactoring, or AI-assisted change. Large changes should be divided into inspectable units.

- Inspection identifier and date.
- Component / repository / branch / commit or pull request.
- Author and reviewers.
- Inspection purpose.
- Files or lines included.
- Explicit exclusions.
- Expected inspection duration and meeting timebox.

3.2 Confirm Entry Criteria

The inspection should not start until the work product is sufficiently stable for review. Entry criteria prevent time being wasted on incomplete or unbuildable code.

- Code compiles/builds or the build status is known and documented.
- Automated tests relevant to the change have been executed, or missing tests are explicitly identified.
- Requirements, user story, defect description, acceptance criteria, or design intent are available.
- Changed files and scope are clearly identified.
- Known limitations or unresolved dependencies are declared.
- AI-assisted code is identified when the organization requires that information to be tracked.

3.3 Distribute the Inspection Package

Provide reviewers with enough context to prepare independently before the meeting.

- Change summary and inspection objective.
- Link or snapshot of source code / pull request.
- Requirements, acceptance criteria, and relevant design artifacts.
- Unit/integration test results and static-analysis results when available.
- Coding, security, architecture, or project-specific standards applicable to the change.
- Known risks, assumptions, and AI-assistance notes when relevant.

4. Individual Preparation

Each inspector reviews the assigned scope independently before the meeting. Preparation is where most defect discovery should occur. The execution meeting is primarily for consolidating findings, clarifying evidence, and agreeing on dispositions.

Requirements & Intent

- Does the code implement the stated requirement or acceptance criteria?

- Are edge cases, failure modes, and invalid inputs addressed?
- Is behavior consistent with design decisions and interfaces?

Correctness & Logic

- Are conditions, loops, state changes, exceptions, and return values correct?
- Are null/empty/boundary cases handled?
- Could concurrency, ordering, or transaction behavior fail?

Architecture & Design

- Does the change preserve intended layering and responsibility boundaries?
- Are dependencies appropriate and minimal?
- Is duplication masking a missing abstraction or reusable component?
- Are interfaces stable and understandable?

Security & Data

- Is input validated appropriately?
- Are authorization, authentication, secrets, logging, and sensitive data handled safely?
- Could the change introduce injection, insecure deserialization, path traversal, unsafe commands, or information leakage?

Maintainability

- Are names, structure, complexity, coupling, cohesion, and comments appropriate?
- Is error handling explicit and diagnosable?
- Would another engineer understand and modify the code safely?

Testing

- Do tests cover intended behavior and important negative cases?
- Do tests verify behavior rather than implementation details only?
- Are defect fixes protected by regression tests?

AI-Assisted Code

- Was AI-generated or AI-modified code independently validated?
- Did the engineer verify API usage, assumptions, edge cases, licensing/security concerns, and tests?
- Is there evidence that human review occurred rather than accepting generated output verbatim?

Preparation record: Each reviewer should record preparation time, files reviewed, findings, severity, evidence/location, and any question that requires clarification during the inspection.

5. Inspection Execution Meeting

Stage	Procedure
Opening (2-5 min)	Moderator confirms objective, scope, entry criteria, participants, timebox, and whether the session is defect-focused rather than a design workshop.
Context summary (5 min)	Author gives a concise explanation of intended change,

	major design decisions, known risks, and tests performed. Avoid line-by-line presentation of the whole codebase.
Finding review	Reviewers present prepared findings. Each finding is located in the code/work product and supported with evidence. Duplicate findings are merged.
Classification	For each material finding, agree on type, severity, disposition, owner, and required closure evidence.
Open questions	Items requiring deeper design or product decisions are recorded and assigned outside the inspection rather than consuming the review session.
Close	Moderator summarizes critical findings, rework expectations, due dates, verification method, and any coaching follow-up.

5.1 Recommended Finding Classification

Severity	Definition
Critical	Potential severe security, data-loss, safety, production outage, or fundamental correctness risk. Must be resolved before release/merge unless formally accepted by accountable authority.
Major	Material defect, requirement violation, architecture/design problem, insufficient validation, or high maintainability/testability risk.
Minor	Localized defect or quality issue with limited impact; should normally be corrected but does not fundamentally invalidate the change.
Observation	Improvement suggestion, question, or coaching opportunity that is not itself a confirmed product defect.

5.2 Finding / Defect Log - Minimum Fields

- Finding ID
- File / class / method / line or artifact location
- Finding description
- Evidence / violated requirement or criterion
- Category (correctness, architecture, security, testing, maintainability, process, etc.)
- Severity
- Disposition (fix, accept, defer, not a defect, requires decision)
- Owner
- Target date
- Closure evidence / verification result

6. Rework and Closure

The author resolves findings and records the action taken. Closure should be verified at a level proportional to risk. Critical and major findings normally require explicit reinspection or evidence review; minor findings may be verified through code diff, test results, or moderator confirmation.

- Rework should not silently delete findings from the log.
- If a finding is rejected, document the rationale and accountable decision maker.
- If risk is accepted or deferred, record why, by whom, and until when.
- Run relevant automated tests and static analysis after rework.
- Update requirements, design, tests, or documentation when the inspection reveals that the defect was not limited to code.

7. Inspection Report

The inspection report is a concise record of what was reviewed, what was found, what was changed, and whether the work product is ready to proceed. It should not be a transcript of the meeting.

Report Section	Required Content
A. Identification	Inspection ID; project/product; component; repository; branch/commit/PR; author; moderator; inspectors; coach; dates.
B. Scope and Objective	What was inspected, why, and what was excluded.
C. Preparation Data	Preparation time by reviewer, approximate size reviewed, tools/results supplied, and readiness status.
D. Summary of Findings	Counts by severity and category, plus concise description of the most material risks.
E. Detailed Finding Log	Finding-by-finding evidence, disposition, owner, and closure status.
F. Rework and Verification	What changed, tests or checks rerun, and how closure was confirmed.
G. Outcome	Accepted; accepted with follow-up; reinspection required; blocked pending decision.
H. Coaching Observations	Evidence-based observations about engineering competencies and targeted follow-up. Avoid including sensitive personnel judgments in a broadly distributed engineering defect report; use a separate coaching record when appropriate.
I. Follow-Up	Actions, responsible person, target date, and next coaching/review checkpoint.

7.1 Useful Inspection Metrics

Metrics are for process learning and coaching, not for simplistic individual ranking. Interpret them with context and trends over repeated inspections.

Metric	Use
Preparation effort	Total reviewer preparation time.
Inspection effort	Meeting duration x participants, if useful for capacity planning.
Findings by severity/category	Distribution of confirmed defects and observations.
Defect density	Findings relative to inspected size, used cautiously and only within comparable contexts.
Rework cycle time	Time from inspection to verified closure.
Repeat finding rate	Whether similar defects recur across later work products.
Escape indicator	Inspection-related defect types later found in test or production; use for process learning.
Coaching progression	Evidence that targeted behaviors improve across repeated inspections.

8. What the Development Coach Should Observe

The coach observes the engineer across preparation, explanation, inspection response, rework, and follow-up. The coach should distinguish isolated mistakes from repeated competency gaps. A single defect is not automatically evidence of inadequate competency.

Observation Area	What to Observe	Possible Evidence
Understanding of requirements	Can the engineer explain intended behavior, constraints, acceptance criteria, and edge cases? Does the code reflect that understanding?	Code/work product, review comments, test evidence, defect log, explanation, rework diff, repeated-inspection trend.
Design reasoning	Can the engineer justify responsibilities, interfaces, dependencies, abstractions, and tradeoffs?	Code/work product, review comments, test evidence, defect log, explanation, rework diff, repeated-inspection trend.
Implementation discipline	Does the engineer produce readable, maintainable code with appropriate error handling and scope control?	Code/work product, review comments, test evidence, defect log, explanation, rework diff, repeated-inspection trend.
Testing competency	Does the engineer select meaningful tests, including negative/boundary cases, and connect tests to risks and requirements?	Code/work product, review comments, test evidence, defect log, explanation, rework diff, repeated-inspection trend.
Review competency	Can the engineer identify defects in others' work, explain evidence, prioritize risk, and avoid superficial style-only review?	Code/work product, review comments, test evidence, defect log, explanation, rework diff, repeated-inspection trend.
Security awareness	Does the engineer recognize relevant	Code/work product, review

	data, trust-boundary, input, authorization, secret, dependency, and logging risks?	comments, test evidence, defect log, explanation, rework diff, repeated-inspection trend.
Use of evidence	Does the engineer use requirements, tests, metrics, static analysis, logs, and code behavior to support conclusions rather than relying only on opinion?	Code/work product, review comments, test evidence, defect log, explanation, rework diff, repeated-inspection trend.
AI-assisted engineering judgment	When AI is used, does the engineer challenge generated output, verify APIs/assumptions, test behavior, inspect security and licensing concerns, and take personal accountability for the result?	Code/work product, review comments, test evidence, defect log, explanation, rework diff, repeated-inspection trend.
Response to findings	Does the engineer seek to understand findings, distinguish disagreement from defensiveness, and resolve issues with evidence?	Code/work product, review comments, test evidence, defect log, explanation, rework diff, repeated-inspection trend.
Learning and follow-through	Are repeated defect patterns decreasing? Are coaching actions applied in later work? Can the engineer generalize the lesson beyond the inspected code?	Code/work product, review comments, test evidence, defect log, explanation, rework diff, repeated-inspection trend.

8.1 Coaching Assessment Guidance

Prefer evidence statements before ratings. If a competency scale is used, define behavioral anchors.

Example:

Level	Behavioral Anchor
Needs Guided Practice	Requires frequent prompting; misses material risks or cannot explain key reasoning consistently.
Developing	Performs parts independently but shows recurring gaps in evidence, completeness, or risk recognition.
Independent	Performs the activity consistently with appropriate evidence and limited coaching.
Advanced / Coaches Others	Handles complex cases, anticipates risk, explains tradeoffs, and helps others improve.

8.2 Coach Anti-Patterns to Avoid

- Turning the inspection into a lecture while the engineer remains passive.
- Scoring an engineer from a single defect or single inspection.
- Confusing coding style preferences with product defects.
- Making coaching notes without observable evidence.
- Fixing the code for the engineer instead of observing reasoning and rework.
- Treating AI use itself as either competence or incompetence; evaluate validation behavior and results.

- Publishing sensitive coaching judgments in the general defect log when a separate coaching record is more appropriate.

9. Quick Inspection Checklist

- ☐ Scope and objective defined
- ☐ Entry criteria met
- ☐ Review package distributed
- ☐ Independent preparation completed
- ☐ Requirements/design/test context available
- ☐ Findings recorded with location and evidence
- ☐ Severity/disposition agreed
- ☐ Owners and target dates assigned
- ☐ Rework completed
- ☐ Major/critical findings verified
- ☐ Inspection outcome recorded
- ☐ Coach observations captured separately where appropriate
- ☐ Follow-up checkpoint scheduled

10. Reusable Code Inspection Report Template

Inspection ID	
Project / Product	
Component / Module	
Repository / Branch / Commit / PR	
Author / Engineer	
Moderator / Inspection Lead	
Inspectors	
Development Coach	
Inspection Date	
Reinspection / Closure Date	

Inspection Objective and Scope

Objective:

Included files / artifacts:

Excluded scope:

Applicable requirements / standards / design references:

Preparation Summary

Reviewer	Preparation Time	Files/Size Reviewed	Tools/Checks	Findings Prepared

Finding Log

ID	Location	Finding / Evidence	Category	Severity	Disposition / Owner	Closure

Inspection Summary

Critical Findings	
Major Findings	
Minor Findings	
Observations	
Overall Outcome	

Rework and Verification

Summary of rework:

Tests/checks rerun:

Verification method:

Remaining accepted/deferred risk:

Coach Observation Record

Competency / Behavior	Observed Evidence	Assessment / Gap	Coaching Action / Follow-Up

Sign-Off / Closure

Moderator / Lead: _____ Date: _____

Author / Engineer: _____ Date: _____

Development Coach (if applicable): _____ Date: _____

Appendix A. Examples of Well-Written Findings

Correctness: OrderService.java: calculateTotal() does not handle an empty discount list. Current implementation can throw a NullPointerException when discounts are absent. Acceptance criterion AC-4 requires checkout to succeed when no discount applies. Severity: Major.

Testing: The defect fix changes retry behavior but adds no regression test for the original failure condition. Add a test that reproduces the timeout sequence and verifies the retry limit. Severity: Major.

Security: User-supplied filename is concatenated into a filesystem path without normalization or allow-list validation. This can permit traversal outside the intended directory. Severity: Critical/Major depending on reachable context.

AI validation: The generated client call references an API method not present in the project dependency version. The code compiles only after an unrelated local dependency override. Engineer should verify the supported API against the actual dependency and add a build/test check. Severity: Major.

End of Guide